

PROYECTO: JUDICIAL SCHEDULER AI AGENT (JSA-Core)

1. Visión del Producto

Desarrollar una aplicación de escritorio para Windows que automatice la gestión, asignación y optimización de turnos de jueces en un tribunal. La aplicación no es una herramienta pasiva; integra un ****Agente de IA**** que actúa como un "Jefe de Operaciones", dialogando con el usuario, proponiendo cronogramas, explicando decisiones lógicas y aprendiendo de las preferencias históricas.

2. Arquitectura del Sistema (Híbrida)

El sistema combina dos cerebros:

- A. ****Motor Lógico Determinista (The Solver):**** Escrito en Python (usando Google OR-Tools o similar). Se encarga de las "Hard Constraints" (matemáticas puras, reglas inquebrantables). Asegura que el cronograma sea legal y válido.
- B. ****Motor Semántico (The Agent):**** Basado en LLM (OpenAI API). Se encarga de la interfaz, la interpretación de intenciones del usuario ("tuneo"), la explicación de conflictos y el manejo de la memoria histórica.

3. Flujo de Trabajo Principal

1. ****Input:**** El usuario carga Excels (Jueces, Ausencias, Roles Fijos).
2. ****Procesamiento:**** El Agente analiza la data y pregunta si hay cambios en las reglas para este ciclo.
3. ****Generación:**** El Agente invoca al Solver. El Solver devuelve una matriz matemática.
4. ****Validación/Explicación:**** El Agente presenta el plan al usuario: "He generado el ciclo. Noté que el Juez X tiene mucha carga, pero es inevitable por sus licencias. ¿Apruebas?".
5. ****Aprendizaje:**** Al aprobar, el ciclo se guarda en DB SQL (estructurado) y las notas de contexto en DB Vectorial (preferencias).
6. ****Output:**** Generación de Excel final formateado.

4. Stack Tecnológico Sugerido

- * ****Backend Logic:**** Python 3.10+
- * ****Solver:**** Google OR-Tools (Constraint Programming).
- * ****Data Handling:**** Pandas, OpenPyXL.
- * ****AI Framework:**** LangChain (para orquestar herramientas).
- * ****Database:**** SQLite (Relacional) + ChromaDB (Vectorial/Memoria).
- * ****UI (Opción recomendada):**** Electron + React (conectado a Python vía API local) o PyQt6 (si se desea 100% Python nativo).